Recap-1
The Compilers' Front End

PART I: Regex → NFA → (Min) DFA

Deterministic Finite Automata

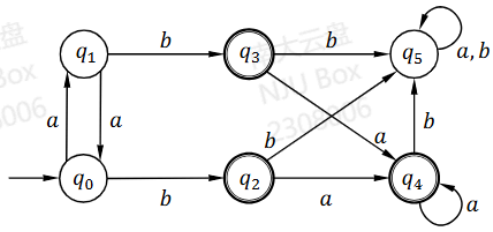
- A DFA is a five-tuple: $(Q, \Sigma, \delta, q_0, F)$
  - Q: A finite set of states
  - Σ: A finite set of input characters, i.e., an alphabet
  - $\delta: Q \times \Sigma \longmapsto Q$: Transition function, e.g., $\delta(q, a) = q'$
  - $q_0 \in Q$: The start state
  - $F \subseteq Q$: A finite subset of final states

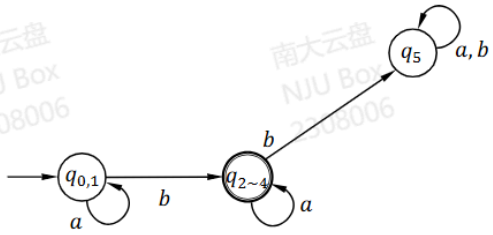
- An extension of transition: $\delta^*: Q \times \Sigma^* \longmapsto Q$
  - $\delta^*(q, abba) = q'$; $\delta^*(q, \epsilon) = q$;

南大云盘 NJU Box 2308006

DFA Minimization

- Minimizing DFA can improve the efficiency of computation

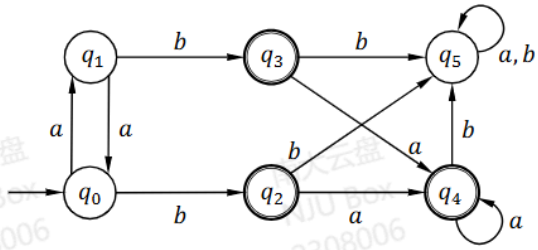


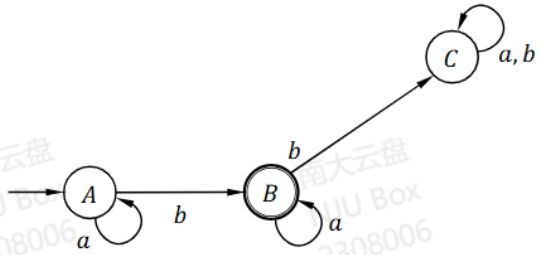


Step 1	$\{q_0, q_1, q_5\}, \{q_2, q_3, q_4\}$	Distinguish final and non-final states
Step 2	$\{q_0, q_1\}, \{q_5\}, \{q_2, q_3, q_4\}$	$\delta(q_0, a/b)$ and $\delta(q_1, a/b)$ are in the same set, but $\delta(q_5, a/b)$ not
Step 3	$\{q_0, q_1\}, \{q_5\}, \{q_2, q_3, q_4\}$	The result does not change and the algorithm completes

DFA Bi-Simulation

- Checking the equivalence of the DFAs, i.e., $L(M_1) = L(M_2)$





State Pairs	a	b	
(q_0, A)	(q_1, A)	(q_2, B)	M_1 reaches final state if and only if M_2 reaches final state
(q_1, A)	(q_0, A)	(q_3, B)	
(q_2, B)	(q_4, B)	(q_5, C)	
(q_3, B)	(q_4, B)	(q_5, C)	
(q_4, B)	(q_4, B)	(q_5, C)	
(q_5, C)	(q_5, C)	(q_5, C)	

Definition of ~~DFA~~NFA

- A ~~DFA~~NFA is a five-tuple: $(Q, \Sigma \cup \{\epsilon\}, \delta, q_0, F)$
  - Q: A finite set of states
  - Σ: A finite set of input characters, i.e., an alphabet
  - $\delta: Q \times (\Sigma \cup \{\epsilon\}) \longmapsto \cancel{Q} 2^Q$: Transition function, e.g., $\delta(q, a) = \{q', q''\}$
  - $q_0 \in Q$: The start state
  - $F \subseteq Q$: A finite subset of final states

From NFA to DFA

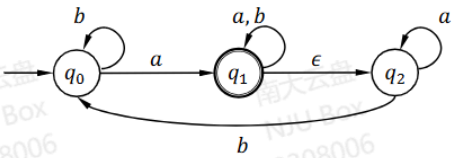
- Given an NFA $M = (Q, \Sigma \cup \{\epsilon\}, \delta, q_0, F)$
  - **Step 1:** initialize a DFA with a start state $\{ q_0 \}$, which is a set of NFA states
  - **Step 2:** for each DFA state $\{ q_i, q_j, \dots, q_m \}$, and char in the alphabet, $a \in \Sigma$

$$\text{let } S = \bigcup \begin{cases} \delta^*(q_i, a \in \Sigma) \\ \delta^*(q_j, a \in \Sigma) \\ \quad \dots \\ \delta^*(q_m, a \in \Sigma) \end{cases}$$

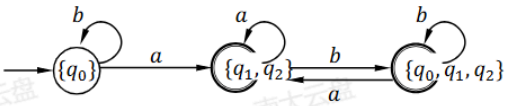
For any input string, the DFA and the NFA reach the final state at the same time, thus DFA=NFA

  - **Step 3**: add transition $\delta(\{ q_i, q_j, \dots, q_m \}, a) = S$ in the DFA

From NFA to DFA



Because the DFA states consist of sets of NFA states, an *n*-state NFA may be converted to a DFA with at most 2^n states. For every *n*, there exist *n*-state NFAs such that every subset of states is reachable from the initial subset, so that the converted DFA has exactly 2^n states, giving $\Theta(2^n)$ worst-case time complexity.



When converting an NFA to a DFA, there is no guarantee that we will have a smaller DFA.

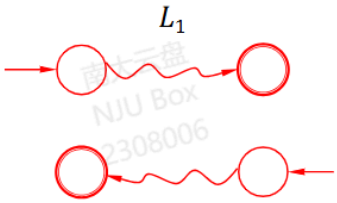
Regular Language

- Take a DFA $M = (Q, \Sigma, \delta, q_0, F)$, language can be accepted by the DFA is written as $L(M) = \{w \in \Sigma^*: \delta^*(q_0, w) \subseteq F\}$
- Take a NFA $M = (Q, \Sigma \cup \{\epsilon\}, \delta, q_0, F)$, language can be accepted by the NFA is written as $L(M) = \{w \in \Sigma^*: \delta^*(q_0, w) \cap F \neq \emptyset\}$

L is a regular language if there's a DFA/NFA M such that $L = L(M)$

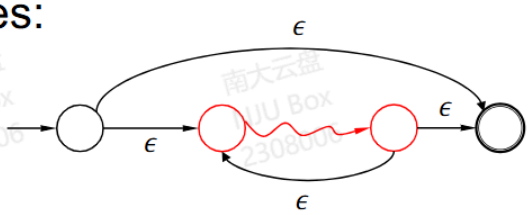
Closure Properties

- Given two regular languages, L_1 and L_2, the following are also regular languages:
  - $L_1 \cup L_2$
  - $L_1 L_2$
  - L_1^R
  - L_1^*
  - $\overline{L_1}$
  - $L_1 \cap L_2$



Closure Properties

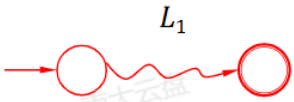
- Given two regular languages, L_1 and L_2, the following are also regular languages:
  - $L_1 \cup L_2$
  - $L_1 L_2$
  - L_1^R
  - L_1^*
  - $\overline{L_1}$
  - $L_1 \cap L_2$



南大云盘 NJU Box 2308006

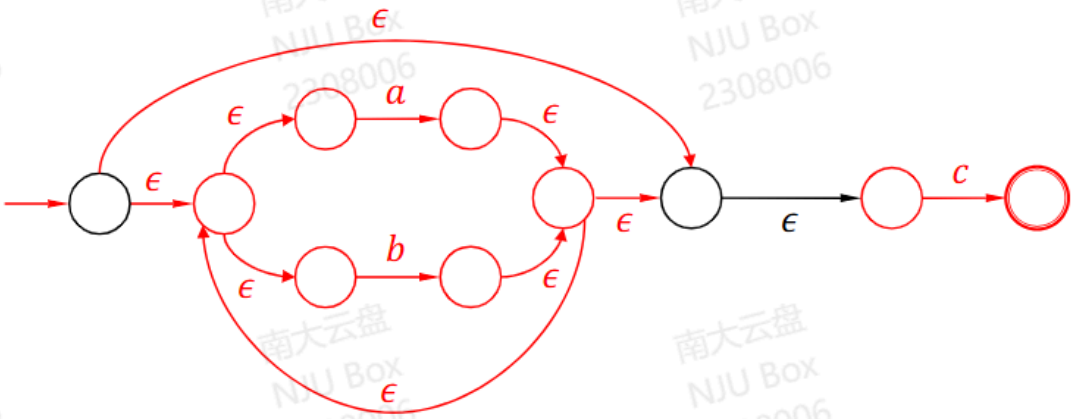
Closure Properties

- Given two regular languages, L_1 and L_2, the following are also regular languages:
  - $L_1 \cup L_2$
  - $L_1 L_2$
  - L_1^R
  - L_1^*
  - $\overline{L_1}$
  - $L_1 \cap L_2$



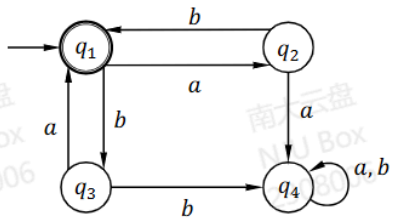
Example

- Build the NFA for the regex: $(a|b)^*c$
- $L\big((a|b)^*c\big) = \big(L(a|b)\big)^* L(c) = \big(L(a) \cup L(b)\big)^* L(c)$



DFA to Regex

- (2) Any ~~regular language~~DFA can be expressed by a regex



$q_1 = \epsilon \mid q_2 b \mid q_3 a$

$q_2 = q_1 a$

$q_3 = q_1 b$

$q_4 = q_2 a \mid q_3 b \mid q_4 a \mid q_4 b$

Variable Elimination

$q_1 = \epsilon \mid q_1 ab \mid q_1 ba$

Associativity

$q_1 = \epsilon \mid q_1 (ab|ba)$

Arden's Theorem:
R = QP is the solution of R = Q | RP*

$q_1 = \epsilon\ (ab|ba)* = (ab|ba)*$

PART II: CFG and Parsing

Context Free Grammar (CFG)

- A context-free grammar is a tuple $G = (N, T, S, P)$
  - N: a finite set of non-terminals
  - T: a finite set of terminals, such that $N \cap T = \emptyset$
  - $S \in N$: start non-terminals
  - P: production rules in the form of $A \to a$, where $A \in N$ and $a \in (N \cup T)^*$

Eliminating Left-Recursion

- A grammar is left-recursive if it has a non-terminal A such that there is a derivation $A \Rightarrow^+ A\alpha$
- **Example:** $A \to A\alpha \mid \beta$ is left-recursive, can be transformed into

$\beta\alpha^*$

$$\boxed{\begin{array}{l} A \to \beta A' \\ A' \to \alpha A' \mid \epsilon \end{array}}$$

Eliminating Left-Recursion

- A grammar is left-recursive if it has a non-terminal A such that there is a derivation $A \Rightarrow^+ A\alpha$
- **Example:** $A \to A\alpha \mid \beta$ is left-recursive, can be transformed into

$$\boxed{\begin{array}{l} A \to \beta A' \\ A' \to \alpha A' \mid \epsilon \end{array}}$$

$$\boxed{A \to A\alpha_1 \mid A\alpha_2 \mid \cdots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \cdots \mid \beta_n}$$

$$\boxed{\begin{array}{l} A \to \beta_1 A' \mid \beta_2 A' \mid \cdots \mid \beta_n A' \\ A' \to \alpha_1 A' \mid \alpha_2 A' \mid \cdots \mid \alpha_m A' \mid \epsilon \end{array}}$$

Predictive Parsing

- Predictive parsers are recursive descent parser w/o backtracking
- LL(1)
  - L: scanning input from left to right
  - L: leftmost derivation
  - 1: Using one input symbol of lookahead at each step

First()

- $\text{FIRST}(\alpha)$:
  - A set of terminals that α may start with
  - If X is a terminal, $\text{FIRST}(X) = \{X\}$
  - If $X \to \epsilon$ is a production, $\epsilon \in \text{FIRST}(X)$
  - If $X \to Y_1 Y_2 \ldots Y_k$, $\epsilon \in \bigcap_{j=1}^{i-1} \text{FIRST}(Y_j) \wedge a \in \text{FIRST}(Y_i) \Rightarrow a \in \text{FIRST}(X)$

$X \to Y_1 Y_2 \ldots Y_{i-1} Y_i \ldots Y_k \to Y_i \ldots Y_k$

First()

- **Example**: $S \to c\ A\ b;\ \ A \to a\ b \mid a$
- $\text{FIRST}(a) = \{a\}$
- $\text{FIRST}(b) = \{b\}$
- $\text{FIRST}(c) = \{c\}$
- $\text{FIRST}(S) = \{c\}$
- $\text{FIRST}(A) = \{a\}$

- $\text{FOLLOW}(\alpha)$:
  - A set of terminals that can appear immediately to the right of α
  - $\$ \in \text{FOLLOW}(S)$, where $\$$ is string's end marker, S the start non-terminal
  - $A \to \alpha B \beta \Rightarrow \text{FIRST}(\beta) \backslash \{\epsilon\} \subseteq \text{FOLLOW}(B)$
  - $A \to \alpha B$ or $A \to \alpha B \beta$ where $\epsilon \in \text{FIRST}(\beta) \Rightarrow \text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)$

A or αB	...

- **Note: repeat the procedure until fixed point!**

Follow()

- **Example**: $S \to c\ A\ b;\ \ A \to a\ b \mid a$
- $\text{FOLLOW}(S) = \{\$\}$
- $\text{FOLLOW}(A) = \{b\}$

- **Example**: $S \to c\ A\ A;\ \ A \to a\ b \mid a$
- $\text{FOLLOW}(S) = \{\$\}$
- $\text{FOLLOW}(A) \supseteq \text{FIRST}(A) \backslash \{\epsilon\} = \{a\}$
- $\text{FOLLOW}(A) \supseteq \text{FOLLOW}(S) = \{\$\}$

Predictive Parsing Table

- To build a parsing table $M[A, a]$, for each $A \to \alpha$
  - $\forall a \in \mathrm{FIRST}(\alpha): M[A, a] = A \to \alpha$
  - $\epsilon \in \mathrm{FIRST}(\alpha) \Rightarrow \forall b \in \mathrm{FOLLOW}(A): M[A, b] = A \to \alpha$

Recap: Three-Address Code

- Provide a standard representation, not as flexible as the source

- At most one operator on the right side of each instruction
- At most three **addresses/variables** in an instruction
  - $\boxed{x} = \boxed{y}\ \mathbf{op}\ \boxed{z}$

Variables or constants;
Variables are from the source code or created by the compiler

南大云盘 NJU Box 2308006

Three-Address Code

- do i = i + 1; while (a[i + 2] < v);

```
L:   t1 = i + 1
     i = t1
     t2 = i + 2
     t3 = a [ t2 ]
     if t3 < v goto L
```

Symbolic Labels

```
100:  t1 = i + 1
101:  i = t1
102:  t2 = i + 2
103:  t3 = a [ t2 ]
104:  if t3 < v goto 100
```

Numeric Labels

- Implementation methods: *quadruples*, *triples*, etc.

Static Single-Assignment

- **Feature 1**: Every variable has only one definition
- **Feature 2**: Using φ to merge definitions from multi paths
- => Direct def-use chains

(a) Three-address code. (b) Static single-assignment form.

Static Single-Assignment

- **Feature 1**: Every variable has only one definition
- **Feature 2**: Using φ to merge definitions from multi paths
- => Direct def-use chains

```
if ( flag ) x = -1; else x = 1;
y = x * a;
```

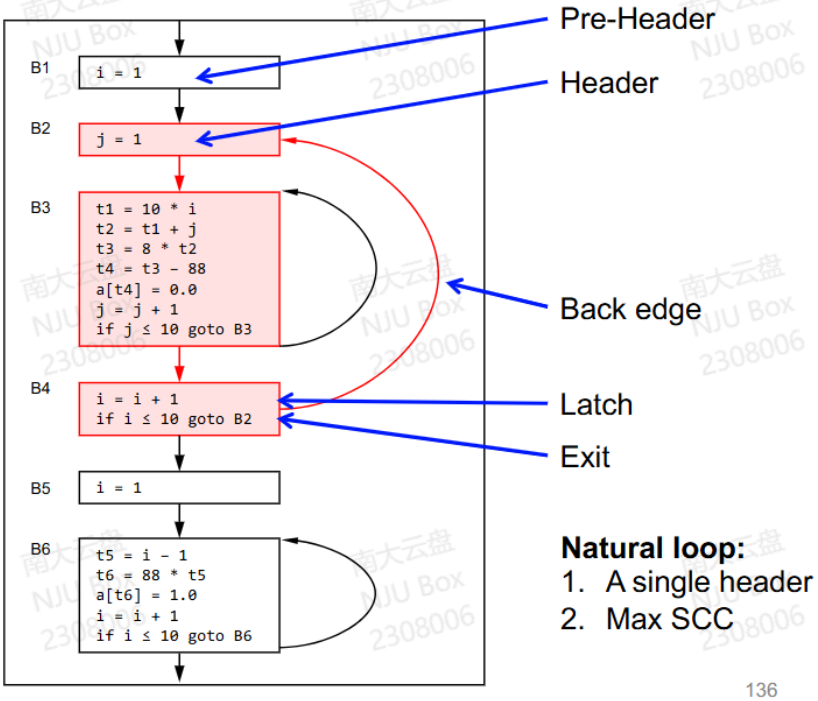
$$\text{if (flag) } x_1 = -1; \text{ else } x_2 = 1;$$

$$x_3 = \varphi(x_1, x_2);$$

$$y = x_3 * a$$

Dominance

- What does that mean for a single header?
- The header **<u>dominates</u>** all blocks in the loop.
  - A **dom** B
  - if all paths from Entry to B goes through A



Natural loop:
1. A single header
2. Max SCC

Post-Dominance

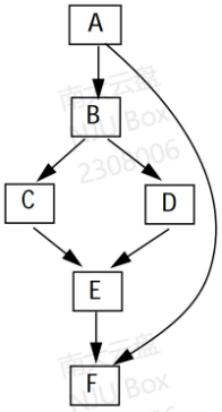
- A **dom** B
  - if all paths from Entry to B goes through A
- A **post-dom** B
  - if all paths from B to Exit goes through A
- Strict (post-)dominance – A (post-)**dom** B but A ≠ B
- Immediate dominance – A strict-dom B, but there's no C, such that A strict-dom C, C strict-dom B

控制依赖可能不会考

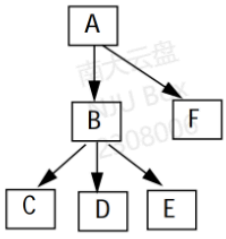
- Block B is control dependent on Block A if and only if
  - B post-dominates a successor of A
  - B does not post-dominates all successors of A

- DF(B) = { … }
  - The immediate successors of the blocks dominated by B
  - Not strictly dominated by B

Block	A	B	C	D	E	F
Dominance Frontier						



Control Flow Graph

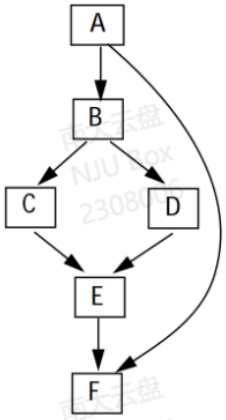


Dominator Tree

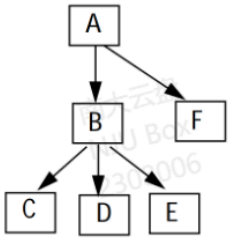
Iterated Dominance Frontier (IDF)

- Iterated DF of Bset
  - DF_1 = DF(Bset); Bset = Bset ∪ DF_1
  - DF_2 = DF(Bset); Bset = Bset ∪ DF_2
  - ……
  - until fixed point! (i.e., $DF_n = DF_{n-1}$)
- IDF({A, B, C}) = { ? }

Block	A	B	C	D	E	F
Dominance Frontier	{ }	{F}	{E}	{E}	{F}	{ }



Control Flow Graph



Dominator Tree

南大云盘 NJU Box 2308006